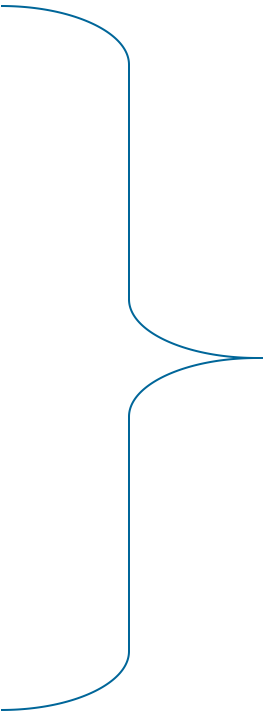




第4章 JSP内置对象

- out对象
- request对象
- response对象
- session对象
- application对象
- page对象
- pageContext对象
- config对象
- exception对象



JSP中共9种内置对象



学习目标

- 熟练掌握JSP的9种内置对象及其常用方法
- 熟练使用request对象获取用户请求信息
- 熟练使用response对象处理响应信息
- 熟练使用session对象实现购物车功能
- 熟练使用application对象实现网站计数器
- 了解Cookie的概念和用处，能使用Cookie对象实现自动登录功能



内置对象概述

- JSP提供了一些由容器实现和管理的内置对象，在JSP页面中可以**直接使用**，不需要实例化，通过存取这些内置对象实现与JSP页面的Servlet环境的相互访问。
- 在JSP中一共提供了**九个内置对象**：**out**、**request**、**response**、**session**、**application**、**pageContext**、**config**、**page**和**exception**。
- 由于JSP是构建在Servlet基础之上的，从本质上讲，JSP的每个内置对象都和Java Servlet API 包中的类相对应，在服务器运行时自动生成。



内置对象概述

对象名	类型	对象说明
out	<code>javax.servlet.jsp.JspWriter</code>	HTML标准输出
request	<code>javax.servlet.http.HttpServletRequest</code>	封装请求信息
response	<code>javax.servlet.http.HttpServletResponse</code>	封装响应信息
session	<code>javax.servlet.http.HttpSession</code>	封装会话信息
application	<code>javax.servlet.ServletContext</code>	封装应用信息
pageContext	<code>javax.servlet.jsp.PageContext</code>	封装当前JSP页面的上下文信息
exception	<code>java.lang.Throwable</code>	封装异常处理信息
config	<code>javax.servlet.ServletConfig</code>	封装JSP页面的Servlet配置信息
page	<code>java.lang.Object</code>	如同Java中的this



1、out对象

- out对应的接口为`javax.servlet.jsp.JspWriter`。
- 作用:主要是向浏览器输出数据,也可以通过out对象对缓冲区进行操作。
- 除了直接使用内置对象以外,我们还可以使用`pageContext.getOut()`方法获取out对象。
- JSP程序利用out对象将网页内容输出时,都要和客户端做一次连接,并且会为此消耗不少的资源;
- 因此可以将要输出的内容放在一个固定大小的缓冲区中,等到缓冲区满时再一次将内容送往客户端,这就要将`autoFlush`设为`true`,否则缓冲区满时将产生`IOException`错误。



out对象主要方法

- out对象向浏览器输出数据的常用方法如下：
 - `public abstract void print()`
 - 显示各种类型的数据
 - `public abstract void println()`
 - 分行显示各种类型的数据
 - `public abstract void newLine()`
 - 输出换行符
 - 注意: `out.println(expression)`与`<%=expression %>`等价
 - 例子: `table.jsp`

一个参数: `boolean`、`char`、`int`、`long`、`float`、`double`、`java.lang.String`、`java.lang.Object`, 系统会自动转换为字符串



out对象主要方法

- out对象对缓冲区进行操作常用方法：
 - public abstract void **clear()**, 清空缓冲区中的内容, 如果缓冲区中的数据已经被刷新到客户端, 则会引发IO异常。
 - public abstract void **clearBuffer()**, 清空缓冲区中的内容, 即使缓冲区中的数据已经被刷新到客户端, 也不会引发IO异常。
 - public abstract void **flush()**, 刷新缓冲区中的数据到客户端。
 - public abstract void **close()**, 刷新缓冲区, 并关闭输出流。
 - public int **getBufferSize()**, 返回缓冲区的大小, 单位字节。
 - public abstract int **getRemaining()**, 返回缓冲区剩余空间的大小, 单位字节。
 - public boolean **isAutoFlush()**, 返回autoFlush的取值。
- 例子: out.jsp



2、request对象

- 是一个`javax.servlet.http.HttpServletRequest`对象。
- 封装了浏览器的请求信息；
- 通过request对象可以获取用户的请求信息；
- 提供了获取cookie、header和session等对象的数据的方法。
- 请求信息的内容包括请求的头信息（Header）、系统信息（比如：编码方式）、请求的方式（比如：GET或POST）、请求的参数名称和参数值等信息



request对象的主要方法

- 1. 获取请求头部信息的方法

- String `getHeader(String name)` , 返回指定头部名称的字符串类型的信息值。
- Enumeration<String> `getHeaderNames()` , 返回所有头部信息的名称。
- Enumeration<String> `getHeaders(String name)` , 返回指定头部名称的所有信息值。
- int `getIntHeader(String name)` , 返回指定头部名称的整型信息值。
- long `getDateHeader(String name)` , 返回指定头部名称的代表日期对象的长整型信息值。

- 例子: header.jsp



request对象的主要方法

- 2. 获取请求参数的方法
 - String `getParameter(String name)` 获取name的参数值。
 - Enumeration `getParameterNames()` 获取所有参数的名称。
 - String[] `getParameterValues(String name)` 获取name的所有参数值。
 - Map `getParameterMap()` 取得一个要求参数的Map。



request对象的主要方法

• 3. 其他方法

- Object getAttribute(String name), 返回指定名称的属性值, 如果指定的属性不存在, 则返回null。
- void setAttribute(String name, Object o), 在请求对象中存储名称为name, 值为o的属性。
- void removeAttribute(String name), 删除指定名称的属性。
- String getContextPath(), 返回请求URI中表示请求上下文的部分。
- Cookie[] getCookies(), 以数组形式返回客户端发送的所有Cookie对象。
- String getMethod(), 获取HTTP请求的方式, 例如: GET, POST或PUT。
- HttpSession getSession(), 获取Session对象。
- void setCharacterEncoding(String encoding), 设置请求体的字符编码方式, 用来解决传递非英文字符所出现的乱码问题。



request对象的主要方法

- 项目1：读取用户的注册信息
 - 注册页面reg.html中表单核心代码
 - `<form method= “post” action= “reg.jsp” >`
 - 服务器处理页面reg.jsp利用request对象获取用户注册的信息
 - 单值参数 `request.getParameter( “参数名” )`
 - 多值参数 `request.getParameterValues( “参数名” )`
 - 注意问题：
 - 支持中文的页面编码（提交前后一致, utf-8）
 - 处理表单提交的中文乱码问题：
`request.setCharacterEncoding( “utf-8” )`



3、response对象

- response对象作用：封装JSP处理数据后产生的结果，并将其传回到客户端响应客户端的请求。
- 对应的接口为javax.servlet.http.HttpServletResponse接口。
- response对象用来提供给客户端浏览器的参考信息，比如响应的头，响应的本体(HTML文本内容)以及服务端的状态码信息。
- JSP会根据客户端的请求建立一个默认的response对象
- 提供了几个用于设置送回浏览器的相应方法。



response对象的主要方法

● 1. 与响应头信息相关的方法

- void **addDateHeader**(String name, long date), 通过指定的名字和值, 新增Date类型的响应头信息。
- void **addHeader**(String name, String value), 通过指定的名字和值, 新增String类型的响应头信息。
- void **addIntHeader**(String name, int value), 通过指定的名字和值, 新增int类型的响应头信息。
- void **setDateHeader**(String name, long date), 通过指定的名字和值, 设置Date类型的响应头信息。
- void **setHeader**(String name, String value), 通过指定的名字和值, 设置String类型的响应头信息。
- void **setIntHeader**(String name, int value), 通过指定的名字和值, 设置int类型的响应头信息。
- String **getHeader**(String name), 返回指定头部名称的字符串类型的响应头信息, 如果name指定的响应头信息未被设置, 则返回null。
- Collection<String> **getHeaderNames**(), 返回所有响应头部名称的集合。
- Collection<String> **getHeaders**(String name), 返回指定头部名称的所有响应头信息的集合。



response对象的主要方法

- 1.与响应头信息相关的方法
 - 典型案例:使用response对象实现页面刷新:
 - setHeader.jsp核心代码
 - response.setHeader("Refresh","3;url=setHeader1.jsp");



response对象的主要方法

- 2. 设定状态显示码的方法
 - void sendError(int sc) 传送状态码(status code)
 - void sendError(int sc, String msg) 传送状态码和错误信息
 - ◆ 用给定的状态码发给客户端一个错误响应。
 - void setStatus(int sc) 设置状态码



response对象的主要方法

了解HTTP状态码（补充）：

1. 状态码分类

HTTP状态码由三位数字组成，根据首位数字可以分为五类：

1xx（信息性状态码）：表示接收的请求正在处理。

2xx（成功状态码）：表示请求正常处理完毕。

3xx（重定向状态码）：需要进一步的操作以完成请求。

4xx（客户端错误状态码）：表示请求包含错误或无法进行。

5xx（服务器错误状态码）：表示服务器处理请求出错。



response对象的主要方法

了解HTTP状态码（补充）：

2. 常见状态码

200 OK: 请求成功，服务器返回客户端请求的数据。

201 Created: 请求成功并且服务器创建了新的资源。

204 No Content: 请求成功，但服务器没有返回任何内容。

301 Moved Permanently: 永久重定向，资源已永久分配新的 URL。

302 Found: 临时重定向，资源临时被分配了新的 URL。

304 Not Modified: 自上次请求后，请求的资源未被修改，可以使用缓存的数据。

400 Bad Request: 服务器无法理解请求的格式，客户端不应该尝试再次使用相同的内容进行请求。

401 Unauthorized: 请求未授权，请求要求用户的身份验证。

403 Forbidden: 服务器理解请求客户端的请求，但是拒绝执行此请求。

404 Not Found: 在服务器上没有找到请求的资源。没有信息告诉用户这种情况是临时的还是永久的。

500 Internal Server Error: 服务器遇到了不知道如何处理的情况。

502 Bad Gateway: 服务器作为网关或代理，从上游服务器收到无效响应。

503 Service Unavailable: 服务器不可用，可能是由于维护或者负载过重。



response对象的主要方法

- 3. 用来重定向的方法

- `void sendRedirect(String url)` 将页面重定向到任意指定的地址（不限于本站点地址）
- `String encodeRedirectURL(String url)`，对指定的重定向地址（加入会话ID）进行编码
- 注意：重定向至少两次请求，不共享数据



response对象的主要方法

- 3. 用来重定向的方法
 - 实现页面重定向例子: `sendRedirect.jsp`
 - 核心代码
 - `response.sendRedirect(“sendRedirect1.jsp?name=Jack”);`
- 注意事项:
 - 页面编码;
 - 重定向过程中的参数传递



常用方法（一）

- `public abstract void sendRedirect(String url)`
 - 将对用户端的响应重新定向到指定的url上。
 - 这个方法必须在响应被提交之前调用。调用这个方法后，响应立即被提交。在调用这个方法后，Servlet不会再有更多的输出。
 - 重定向注意事项
 - 浏览器向服务器发送一个请求并收到响应后再次向一个新地址发出请求
 - 重定向可以跳转到任意URL
 - 客户端至少请求两次，地址栏会发生变化
 - 两次请求不共享数据
 - 客户端行为



常用方法（二）

- public abstract PrintWriter **getWriter()**

- 获取输出流，将字符写到浏览器上。

- 例：

```
<%
```

```
java.io.PrintWriter pw=response.getWriter();
```

```
pw.println("hi");
```

```
%>
```



常用方法（三）

- public abstract void **setContentType**(String type)
 - 设置响应的MIME类型
 - 例： **将当前页面转换为word文档**
 - `<% response.setContentType( “application/msword;charset=gb2312” ); %>`



常用方法（四）

- `public abstract void addCookie(Cookie cookie)`
 - 在响应中增加一个指定的cookie。
 - 可多次调用该方法以定义多个cookie。为了设置适当的头域，该方法应该在响应被提交之前调用。



常用方法（五）

- `public abstract String encodeRedirectURL(String url)`
 - 将指定的URL编码，以便在`sendRedirect()`方法中使用。
 - 相对URL不能被接收，会抛出一个`IllegalArgumentException`。
必须给出一个绝对的URL。
 - 所有提供给`sendRedirect`方法的URL都应通过这个方法运行，这样才能确保会话跟踪能够在所有浏览器中正常运行。



常用方法（六）

- public abstract String **encodeURL()**
 - 对包含session ID的URL进行编码。
 - 将指定的URL和session ID一起编码。
 - 用于对超链接和form表单的action属性中设置的URL进行重写。
 - 所有提供给Servlet的URL都应通过这个方法运行，这样才能确保会话跟踪能够在所有浏览器中正常运行。
 - 注意：
 - encodeURL()是**本应用**级别的，encodeRedirectURL()是**跨应用**的。
 - 在使用重定向时，
response.sendRedirect(response.encodeRedirectURL(***))
时一定要使用encodeRedirectUrl（）



常用方法（七）

- 设置浏览器无缓冲：
 - `setHeader("Pragma", "no-cache");`
 - `setHeader("Cache-Control", "no-cache");`
- 设置自动刷新
 - `response.setHeader("Refresh", "30");` // 每隔30秒自动刷新当前页面
 - `response.setHeader("Refresh", "10;URL=http://localhost/new.jsp");` // 10秒后加载新的页面



response案例

- 项目2：模拟用户登录

- 注意：

- 中文编码支持的实现

- 不同页面统一编码为utf-8
 - 表单提交页面setCharacterEncoding ()

- 不同页面间参数的传递

- 表单提交 request.getParameter("参数")
 - 重定向时 url?参数=值 request.getParameter("参数")

- 自动刷新的实现

- response.setHeader("Refresh", "5; url=login.html")



4、session对象

- session对象作用：在会话范围内，记录每个客户端的访问状态，以便于跟踪每个客户端的操作状态
- 所实现的接口：
`javax.servlet.http.HttpSession`。
- 可以在JSP页面中直接使用session对象，也可以通过`pageContext.getSession()` 或 `request.getSession ()`方法重新获取。



session的工作原理

- 客户首次访问服务器的一个页面时，服务器就会为该客户分配一个session对象，同时为该session对象指定一个唯一的ID，并将该ID号发送到客户端并写入到Cookie中，使得客户端与服务器的session建立一一对应关系。
- 当客户继续访问服务上的其他资源时，不再为该客户分配新的session，直到客户端浏览器关闭、超时或调用session的invalidate()方法使其失效，客户端与服务器的会话结束。
- 当客户端重新打开浏览器访问网站时，服务器会重新为客户分配一个session对象，重新分配session id。
- 常见功能：保存用户的登录信息，购物车等



常用方法 （一）

- session的方法主要用于属性操作和会话管理：
 - public void **setAttribute**(String name,java.lang.Object value)
 - 设定指定名字的属性的值，并且将它添加到session对象中，如果这个属性存在，则更改该属性的值。
 - public Object **getAttribute**(String name)
 - 获取指定名字的属性的值，如果该属性不存在，则返回null。
 - public void **removeAttribute**(String name)
 - 删除指定的session属性
 - 若该属性不存在则出现异常



常用方法（二）

- `public void invalidate()`
 - 使session无效
- `public String getId()`: 获取当前的会话ID
 - 每个会话在服务器端存在一个唯一的标识: 会话ID, session对象发送到浏览器的唯一数据就是会话ID。
 - 它一般存储在Cookie中。



常用方法（三）

- `public void setMaxInactiveInterval(int interval)`
 - 设置会话的最大持续时间，单位是秒，负数表明会话永不失效。
- `public int getMaxInactiveInterval()`
 - 获取会话的最大持续时间



常用方法（四）

- 演示session常用方法的例子： session.jsp
 - 设置和获取属性
 - session.setAttribute("属性名","值")
 - session.getAttribute("属性名")
 - 获取会话ID session.getID()
 - 是否新会话 session.isNew()
 - 获取及设置会话持续时间
 - session.getMaxInactiveInterval()
 - session.setMaxInactiveInterval()
 - 创建时间 session.getCreationTime()
 - 最后访问时间 session.getLastAccessedTime()
 - 使会话失效 session.invalidate()



session案例

- 项目3：使用session实现用户登录验证
 - 注意：
 - session在不同网页间的属性操作
 - 中文编码支持
 - 重定向的实现
 - 自动刷新的实现



session案例

• 项目4：使用session实现购物车

– 注意：

- 购物车的实现-注意carts属性
- session在不同网页间的属性操作
 - carts属性，loginUserName属性
- 中文编码支持
- 数据在不同网页间的传递-表单参数，session属性操作
- include指令的使用、page指令的重要属性
- 重定向的实现
- 自动刷新实现



Cookie对象

- **不是JSP内置对象**。其作用：Cookie对象通常用于在浏览器端保存会话过程中的一些参数。当浏览器访问Web服务器（某一站点）时，相应的cookie会自动发送到服务器上。
 - HTTP协议是一种**无连接**的协议（主要说明为什么需要Cookie和session）。
- Cookie的常用用途：
 - 购物车、逐步注册、搜索引擎记录、个性化服务
 - Cookie是站点跟踪特定访问者访问的次数，最后访问的时间以及访问者进入站点的路径；
 - Cookie能够帮助站点统计用户个人资料以实现各种各样的个性化服务。
 - cookie实现自动登录功能，使得用户不需要输入用户名和密码就可以进入曾经浏览的站点。（限定在同一网站）



Cookie

- Cookie是Web服务器保存在用户硬盘上的一段文本。保存路径C:\Documents and Settings\登录用户名\Cookies（因操作系统版本而异）
- Cookie信息保存在客户机上。
- Cookie中，信息的片断以‘名/值’对(name-value pairs)的形式储存。



Cookie

- **cookie:** 是一个小的、已命名的数据元素，服务器使用Set-Cookie头标将之作为HTTP数据包的一部分传送到客户端，实质是一段纯文本信息，保存在客户端指定目录，对同一服务器的后续请求使用一个cookie头标返回到服务器。
 - cookie是与Web站点而不是与具体页面相联系的。
 - cookie是有时效性的，有效期时间以秒为单位。
- 从编程角度看，JSP中的cookie是Java提供的一个类
- **创建cookie:**
 - cookie中带有两个字符串参数代表cookie的名称和值

```
Cookie demoCookie =  
new Cookie("CookieName","CookieValue")
```



设置/获取Cookie属性

返回类型	方法名	完成功能
void	setName (String name)	设置Cookie的名字
String	getName()	返回cookie的名字
void	setValue(String value)	设置Cookie的值
String	getValue()	返回Cookie的值
void	setDomain(String domain)	设置cookie中适用的域名
String	getDomain(String domain)	获取cookie中适用的域名
void	setMaxAge(int second)	设置cookie的存活周期
int	getMaxAge(int second)	获取cookie在失效以前的最大时间，以秒计算
void	setPath(String path);	设置能够读取Cookie的路径
String	getPath(String path);	返回Cookie的适用路径
void	setComment(String msg)	设置cookie的注释
void	setSecure(boolean flag)	设置浏览器是否需要使用安全协议才能读取该cookie



写入Cookie方法

- 对于cookie的写入，要结合response对象实现
- 写入Cookie的主要步骤：
 1. 创建Cookie对象
 2. 设定Cookie的属性（一般设置Cookie的有效期）
 3. 调用response.addCookie(Cookie c)方法将其写入到客户端

<%

```
Cookie c=new Cookie("season", "spring");  
c.setMaxAge(30);    //cookie的有效期为30秒  
response.addCookie(c);
```

%>

%>



读取Cookie方法

- 对于cookie的读取，要结合request对象实现

<%

```
Cookie a[]=request.getCookies();
```

```
//根据设置时的cookie个数决定数组元素个数
```

```
String s1=(String)a[0].getName();
```

```
String s2=(String)a[0].getValue();
```

```
out.println(s1+s2);
```

%>



Cookie的有效期

- 1) 过期属性是按秒为单位记录的，使用正整数
- 2) 负值表示该cookie的生存期是当前浏览器会话
- 3) 零值表示立即删除该cookie
- 如果不设置cookie的有效期，就不能在硬盘上保存cookie的信息，一旦浏览器关闭，cookie信息就消失。
- Cookie有效期的设置必须在`response.addCookie()`方法之前。
- 例子：`cookie1.jsp`和`cookie2.jsp`



Cook i e案例

- 项目5：使用Cook i e实现自动登录功能
 - 分析自动登录实现的关键代码
 - 注意：
 - 中文编码支持
 - 表单使用
 - cookie的读取
 - cookie的写入
 - 通过session的属性操作，实现不同网页间数据的传递
 - request的使用
 - response重定向



cookie与session的区别和联系

- 不同:

- **存放地点**: cookie存放在客户端的硬盘里, 属于离线存放, 而session存放在服务器的内存中。
- **存活时间**: cookie可以长期存放在客户端, 具体的存活时间由setMaxAge()方法所指定的数值决定, session随用户访问服务器而产生, 随客户超时或下线而消失。
- **安全性**: cookie存放在客户端, 可能会被别有用心网站读取, 安全性较差, 而session存放在服务器的内存中, 用户不能修改, 且随客户端浏览器的关闭而消失, 安全性较好。

- **联系**: 不论是cookie还是session内置对象都需要浏览器支持cookie并且没有禁用cookie。



5、application对象

- application对象：负责提供应用程序在服务器运行时的一些全局信息，它在服务器启动时创建，直到服务器关闭时消失，它是实现的是`javax.servlet.ServletContext`。
- 常用方法如下：
 - 有关属性操作的方法
 - `public void setAttribute(String name, Object value)`，在application对象中存储指定名字的属性和值。如果指定的属性名已经存在，则更改这个属性的值。
 - `public Object getAttribute(String name)`，读取application对象中指定名字的属性的值，如果指定的属性名不存在，则返回null。
 - `public Enumeration getAttributeNames()`，获取application对象中所有属性的名字。
 - `public void removeAttribute(String name)`，删除application对象中指定名字的属性，如果指定的属性名不存在，则什么都不做。



application对象的主要方法

- 获取容器相关信息的方法：

- public int getMajorVersion(), 获取Servlet容器支持的Servlet API的主版本号。
- public int getMinorVersion(), 获取Servlet容器支持的Servlet API的主版本号。
- public String getServerInfo(), 获取Servlet容器的名称和版本号。

- 其他方法

- public String getMimeType(String file), 获取指定文件的MIME类型。
- public String getRealPath(String path), 获取指定path的绝对路径。
- public String getServletContextName(), 获取Web应用的名字。
- public void log(String msg), 将指定的信息写入日志文件。



application案例

- 项目6：使用application实现网页访问计数功能
 - 注意
 - application的属性操作
 - 中文编码支持



6、pageContext对象

- JSP利用页面上下文对象pageContext可以获取页面所有相关的属性。
- 通过pageContext对象也可以得到当前页面中所有其他的JSP内置对象(out、request、response、session、application等) 及其所支持的属性。
- 它实现的接口为javax.servlet.jsp.PageContext。



pageContext对象的常用方法

• 1. 用户获得属性的方法:

- `public void setAttribute(String name, Object value)`, 在pageContext对象中存储指定名字的属性和值。如果值为null, 则效果等同于`removeAttribute(String name)`。
- `public void setAttribute(String name, Object value, int scope)`, 在指定范围内存储指定名字的属性和值。如果值为null, 则效果等同于`removeAttribute(String name, int scope)`。
- `public Object getAttribute(String name)`, 读取pageContext对象中指定名字的属性的值, 如果指定的属性名不存在, 则返回null。
- `public Object getAttribute(String name, int scope)`, 读取指定范围内指定名字的属性的值, 如果指定的属性名不存在, 则返回null。
- `public Enumeration getAttributeNamesInScope(int scope)`, 获取指定范围内所有属性的名字。
- `public int getAttributesScope(String name)`, 读取指定名字的属性的所属范围。



pageContext对象的常用方法

- public void **remove**Attribute(String name), 删除pageContext对象中指定名字的属性, 如果指定的属性名不存在, 则什么都不做。
- public void removeAttribute(String name, int scope), 删除指定范围内指定名字的属性, 如果指定的属性名不存在, 则什么都不做。
- public Object **find**Attribute(String name), 依次在pageContext对象, request对象, session对象, application对象中搜索指定名字的属性。如果找到, 则返回属性的值, 如果没有找到, 则返回null。
- 其中scope的取值为javax.servlet.jsp.PageContext类所提供的四个静态常量:

常量名称	说明
PAGE_SCOPE	pageContext对象的属性范围
REQUEST_SCOPE	request对象的属性范围
SESSION_SCOPE	session对象的属性范围
APPLICATION_SCOPE	application对象的属性范围



pageContext对象的常用方法

- 2. 用于获取其他内置对象的方法

Exception getException()	获取当前页面出现的异常（即exception对象），不过此时页面应为错误处理页.
JspWriter getOut()	获取当前页面的输出流,即out对象
Object getPage()	获取当前页面的Servlet对象，即page对象
ServletRequest getRequest()	获取当前页面的请求对象，即request对象
ServletResponse getResponse()	获取当前页面的响应对象，即Response对象
ServletConfig getServletConfig()	获取当前页面的ServletConfig对象，即config对象
ServletContext getServletContext()	获取当前的ServletContext对象，即application对象
HttpSession getSession()	获取当前页面的会话对象，即session对象



7、config对象

- config对象代表当前JSP页面的配置信息，它对应的接口是：
是javax.servlet.ServletConfig。通常情况下，JSP页面无需配置，因此config对象在JSP页面中比较少用，但它在Servlet中的作用比较大，将在相关部分详细介绍。
- config对象的常用方法：
 - public String `getInitParameter(String name)`，获取指定名字的初始化参数的值，如果参数不存在，则返回null。
 - public Enumeration `getInitParameterNames()`，获取所有初始化参数的名字，如果没有初始化参数，则返回一个空的枚举集合。
 - public ServletContext `getServletContext()`，获取Servlet的上下文对象。
 - public String `getServletName()`，获取Servlet对象的名称。



8、page对象

- page对象是JSP页面对应的Servlet类的实现对象，相当于this。
- 实际指的就是JSP页面本身。
- 实际很少使用，因此在此略过。



9、exception对象

- exception对象代表了JSP文件运行时所产生的错误和异常对象；
- 只有在包含`<%@page isErrorPage="true"%>`的页面中才可以使用，在一般的JSP文件中使用该对象会导致无法编译JSP文件。
- 是java.lang.Throwable的一个实例。



exception对象的常用方法

- 返回错误信息
 - `public String getMessage()`
 - `public String toString()`
- 输出详细错误信息：
 - `public void printStackTrace()`
 - `public void printStackTrace(PrintStream ps)`
 - `public void printStackTrace(PrintWriter pw)`



小结

- **out对象**的主要作用是向浏览器输出数据信息，也可以通过out对象操作缓冲区。
- **request对象**封装了浏览器的请求信息，通过request对象的各种方法可以获取客户端以及用户提交的各项请求信息。
- **response对象**封装了服务器端处理请求时生成的响应信息，请求处理完成后，服务器将其返回给客户端。。
- **session对象**是在会话范围内，记录每个客户端的访问状态，以便于跟踪每个客户端的操作。一般需配合cookie使用。其中cookie不是JSP内置对象，但需学会**cookie读写方法**。
- **application对象**负责为JSP页面提供在服务器运行时的一些全局信息，它在服务器启动时创建，直到服务器关闭时消失。
- **pageContext对象**提供了对当前JSP页面相关信息的访问，通过它可以得到其他8种内置对象。



作业

- 结合案例，指出out对象的主要方法。
- 结合案例，列举request对象的主要方法。
- 结合案例，列举说明response对象的主要方法。
- 描述session概念，说明session工作原理。结合案例，给出session的重要方法。
- 如何写入及读取cookie，给出核心代码。
- 如何使用application对象实现网站计数器。
- 结合购物车代码，说明以下关键代码：
 - 网页的中文编码支持？
 - 网页间的数据传递？
 - 刷新页面？
 - 重定向代码？与页面转发的区别？